Cada App con su Arquitectura

P. L. Alvarez Gonzalez
7690-22-5839
Universidad Mariano Gálvez
Seminario de Tecnologías de Información
palvarezg1@miumg.edu.gt

Resumen

El objetivo de este trabajo es explicar de forma clara y práctica cómo se debe elegir la arquitectura de una aplicación dependiendo del tipo de proyecto que se quiera desarrollar, ya que no existe una arquitectura que sirva para todo. Para lograrlo se investigó en publicaciónes web de ingeniería de software y se ordenó la información describiendo las arquitecturas más usadas en la industria como la monolítica, cliente-servidor, n-capas, microservicios, orientada a servicios y basada en eventos. Luego se analizó en qué escenarios conviene usar cada una, tomando en cuenta cosas como el tamaño del equipo, el presupuesto, la escalabilidad que se necesita y el tiempo que se tiene para sacar el proyecto adelante. Como resultado se armó una guía comparativa que ayuda a decidir con criterio en vez de escoger una arquitectura solo porque está de moda. Se concluye que la arquitectura correcta depende del contexto del proyecto y no de una regla fija y que elegir mal desde el inicio puede salir muy caro de corregir después.

Palabras clave: arquitectura de software, microservicios, monolito, escalabilidad, diseño de sistemas.

Desarrollo del tema

Elegir la arquitectura de una aplicación es una de las decisiones más importantes dentro del ciclo de vida del desarrollo de software, porque de ahí depende qué tan fácil sea escalar el sistema, mantenerlo y hacerle cambios en el futuro. Esta decisión se debe tomar después de tener los requisitos del proyecto, ya que la arquitectura debe responder a las necesidades reales de la aplicación y no al gusto personal del desarrollador.

La arquitectura monolítica es aquella donde toda la aplicación, la parte visual, la lógica de negocio y el acceso a datos, vive en un solo bloque de código que se despliega como una sola unidad. Esta arquitectura es ideal para proyectos pequeños, aplicaciones que están empezando o equipos pequeños que necesitan sacar algo funcional rápido, porque es más sencilla de desarrollar, probar y desplegar al inicio. El problema aparece cuando la aplicación crece mucho, porque cualquier cambio pequeño obliga a volver a desplegar todo el sistema completo y eso se vuelve lento y riesgoso.

La arquitectura cliente-servidor separa la aplicación en dos partes: el cliente, que es la interfaz que usa el usuario y el servidor, que procesa la lógica y gestiona los datos. Esta arquitectura es la base de la mayoría de aplicaciones web y móviles actuales y funciona bien cuando se necesita que varios clientes como navegadores, apps móviles se conecten a un mismo servidor central.

La arquitectura n-capas es una mejora de la anterior, donde el sistema se divide en varias capas independientes, normalmente presentación, lógica de negocio y acceso a datos. Esta separación

facilita el mantenimiento porque cada capa se puede modificar sin afectar directamente a las demás, siempre y cuando se respete la comunicación entre ellas. Es una buena opción para aplicaciones empresariales de tamaño medio que necesitan orden y organización, pero que todavía no requieren microservicios.

La arquitectura de microservicios divide la aplicación en varios servicios pequeños e independientes, donde cada uno se encarga de una función específica del negocio y se puede desarrollar, desplegar y escalar por separado. Esta arquitectura conviene cuando la aplicación es grande, cuando varios equipos trabajan en paralelo o cuando se necesita escalar solo una parte del sistema sin tener que escalar todo lo demás. La desventaja es que es más compleja de administrar, porque hay que manejar la comunicación entre servicios, el monitoreo y la coordinación de despliegues.

La arquitectura orientada a servicios es parecida a los microservicios en el sentido de que divide el sistema en servicios, pero se ayuda como en un bus de servicios centralizado que gestiona la comunicación entre ellos. Esta arquitectura es común en empresas grandes que ya tienen varios sistemas antiguos y necesitan que todos se comuniquen entre sí de forma ordenada, sin tener que reescribir todo desde cero.

Por último, la arquitectura basada en eventos organiza el sistema alrededor de eventos que se disparan cuando algo sucede y distintos componentes reaccionan a esos eventos sin depender directamente unos de otros. Esta arquitectura es muy útil en aplicaciones que necesitan procesar información en tiempo real, como notificaciones, plataformas que manejan muchos datos que cambian constantemente.

Para elegir entre todas estas opciones, se deben tomar en cuenta varios factores como el tamaño y la experiencia del equipo de desarrollo, el presupuesto disponible, qué tan rápido se necesita lanzar el proyecto, cuánto va a crecer la aplicación a futuro y qué tan crítico es que el sistema esté disponible todo el tiempo. Un equipo pequeño con poco tiempo probablemente le va a ir mejor con un monolito bien organizado, mientras que una empresa grande con varios equipos y necesidad de escalar por partes va a beneficiarse más de microservicios.

Observaciones y comentarios

Durante esta investigación se pudo notar que muchos proyectos fracasan no porque la arquitectura elegida sea mala en sí misma, sino porque se elige una arquitectura demasiado compleja para el tamaño real del proyecto. Es muy común ver equipos pequeños que arrancan con microservicios desde el día uno solo porque es la tendencia y terminan gastando más tiempo administrando la infraestructura que hacinendo funcionalidades para el usuario. También se observó que muchas empresas migran de monolito a microservicios con el tiempo, conforme el proyecto va creciendo, en vez de empezar directamente con una arquitectura compleja.

Conclusiones

1. No existe una arquitectura perfecta, la elección correcta depende del tamaño del proyecto, del equipo y de los objetivos del negocio.

2. Los proyectos pequeños o que necesitan salir rápido se benefician de arquitecturas simples como el monolito o cliente-servidor.

3. Los proyectos grandes, con varios equipos y necesidad de escalar por partes, se benefician más de microservicios o arquitecturas orientadas a servicios.

4. Elegir una arquitectura más compleja de la que realmente se necesita puede retrasar el proyecto en vez de ayudarlo.

Referencias

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

Sommerville, I. (2016). *Ingeniería de software* (10.ª ed.). Pearson Educación.

Repositorio: https://github.com/PLAlvarez11/seminario-encuesta.git